YJIT

A second JIT compiler ships with Ruby that takes a significantly different approach to compilation. YJIT (yet another Yet Another acronym) is a complete compiler on its own, and as of Ruby 3.2, it is also implemented in Rust. YJIT uses a mechanism called "lazy basic block versioning" for compilation, which means it compiles sections of code based on chunks of code that are smaller than methods, such as loops.

YJIT also takes advantage of runtime type information to optimize compilation for known types—if it sees that a chunk of code is called with integer values, it will optimize the compiled steps for integer values. If the chunk is called often enough with a different type of value, then different compiled code is generated. That's the "versioning" part of basic block versioning—YJIT creates different compiled versions of blocks based on input values. The "lazy" part is that YJIT only generates versions for blocks that it sees on the fly during runtime. The combination means that YJIT can generate fast compiled code without compromising on Ruby's flexibility.

YJIT is installed along with Ruby, provided that you have the Rust language version 1.58 or higher installed on your computer when Ruby is installed. Rust installation instructions can be found at https://www.rust-lang.org/tools/install.

YJIT can be executed as part of the `ruby` runtime by adding `--yjit` to the call:

```
ruby options.foo --yjit
```

YJIT can also be enabled by setting the environment variable `RUBY_YJIT_ENABLE` to true.

Using YJIT can provide a significant performance boost for long-running processes. Because YJIT only compiles methods after they have been repeated several times—the default is 30—you likely won't see a benefit in shorter scripts or in unit tests. As this is being written, the Rust version of YJIT is brand new in the wild, so there aren't a lot of use cases to check, but one early test suggests that performance on benchmarks can double using YJIT.[2]

YJIT has some additional command-line options that tweak the behavior and allow for more debugging information if you are actually working on YJIT itself. Note that some of the debugging information depends on having a version of Ruby compiled to produce YJIT stats.

`--yjit-call-threshold=num`	The number of times a method is called before YJIT optimizes it by compiling. The default is 30.
`--yjit-exec-mem-size=num`	The size, in MiB, of the memory dedicated to executable code. The default is 128.
`--yjit-greedy-versioning`	If set, enables greedy versioning mode, which may increase the size of the compiled code.
`--yjit-max-versions=N`	The maximum number of versions of a compiled block YJIT will generate for different type information. The default is 4.
`--yjit-stats`	If enabled, stats about YJIT usage are displayed at the end of the program.

2. https://www.solnic.dev/p/benchmarking-ruby-32-with-yjit